

Time-Series Database Engine

Design Document

Group 02

Advanced Database Management Systems
Computer Science (Sections A & B) — 4th Semester
Information Technology University, Lahore
Spring 2025

1. Introduction

This document describes the design and implementation of a custom time-series database engine (TSDB) built from scratch in C. The system ingests timestamped floating-point measurements, compresses them using bit-level Gorilla encoding, stores them on disk in a binary chunk format, and answers range queries and aggregations over them via a TCP text protocol.

1.1 Motivation

General-purpose relational databases are ill-suited for time-series data for three reasons. First, writes are almost always append-only: new points arrive strictly after all previously stored points. Second, timestamps are highly regular — sensor data sampled once per second has timestamps spaced exactly 1 second apart. Third, consecutive values tend to be numerically close: a CPU utilisation reading of 45.2% is usually followed by 45.3%, not 99.9%. Our TSDB exploits all three properties.

The two core compression techniques originate from the Gorilla paper (Pelkonen et al., VLDB 2015), the same algorithms used by Prometheus, InfluxDB, and every modern monitoring system:

- **Delta-of-delta encoding** for timestamps: the difference between consecutive deltas is almost always zero for regular metrics.
- **XOR encoding** for values: two close IEEE 754 doubles differ in only a few mantissa bits; XOR-ing them leaves a small block of meaningful bits surrounded by long runs of zeros.

Together these techniques reduce the naive 16-bytes-per-point format to roughly 1.4 bytes per point on real workloads — a $> 10\times$ reduction — while still allowing scans at hundreds of thousands of points per second.

1.2 Scope and Constraints

The system supports flat metric names (letters, digits, dots, underscores), 64-bit Unix timestamps in seconds, and IEEE 754 double values. There is no SQL, no multi-dimensional labels, no transactions, and no concurrency control beyond per-metric locking. Out-of-order PUTs are rejected. The implementation language is C11; no external compression or serialization libraries are used — all bit-level encoding is hand-written.

2. Architectural Overview

The system is divided into three layers that form a strict hierarchy. Higher layers call lower layers but never the reverse.

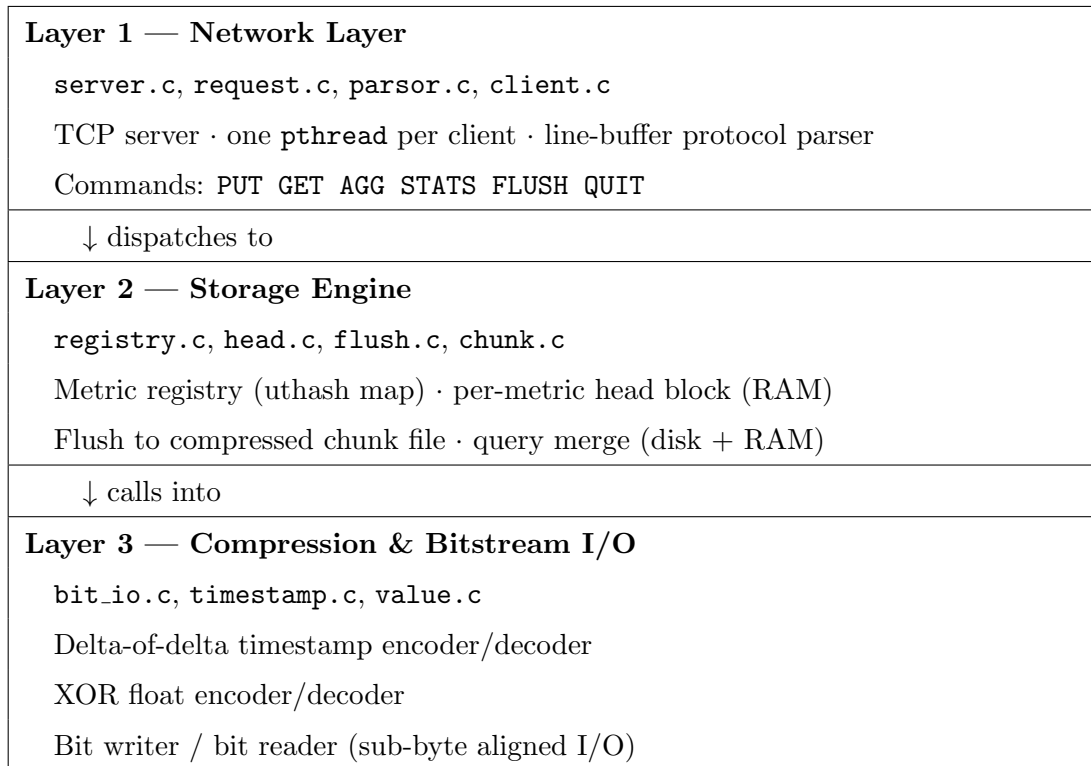


Figure 1: Three-layer architecture

Background threads (WAL, retention, downsampling) sit alongside the storage engine and interact with it exclusively through the `registry_lock` mutex and the public functions of `registry.h`.

3. Network Layer

3.1 TCP Server

The server is created with the standard BSD socket API: `socket` → `setsockopt(SO_REUSEADDR)` → `bind` → `listen(128)` → `accept` loop. For every accepted connection, `createPthreadForUsers` allocates a `ThreadArgs` struct (client socket + data directory path) on the heap, spawns a detached `pthread`, and returns. The detached thread owns its client socket and frees `ThreadArgs` at start. Graceful shutdown is handled via `SIGINT`/`SIGTERM` handlers that set a volatile `bool server_running = false` and close the server socket. The blocked `accept` call then returns `EBADF`, the `accept` loop exits, and the main thread calls `retention_stop`, `downsample_stop`, and `cleanupRegistry` (which flushes all dirty head blocks to disk before freeing memory).

3.2 Protocol Parser

The wire protocol is plain text, one command per line, newline-terminated. Each client thread accumulates incoming bytes into an 8 KB `inbuf`. When a newline is found, the line is extracted and passed to `getRequest`, which tokenizes on whitespace (`parsor.c`) and constructs a `Request` struct with typed fields:

Listing 1: Request struct

```

1 typedef struct {
2     enum { PUT, GET, AGG, STATS, FLUSH, QUIT } type;
3     char    metric[64];
4     long    timestamp;
5     double  value;
6     long    startTimeStamp, endTimeStamp;
7     int     bucketSeconds;
8     char    func[16];
9     int     resultCount;
10 } Request;

```

Parsing is strict: wrong token count returns NULL and the server replies "Invalid Command!\n".

3.3 Command Dispatch

`ProcessRequest` switches on `request->type` and calls the appropriate handler. Each handler is a thin wrapper that extracts fields from `Request` and calls into the storage engine. The response is either a heap-allocated string (for GET/AGG/STATS) or NULL (for PUT/FLUSH, which reply with "ok"). The server sends the response and frees the string.

4. Storage Engine

4.1 Metric Registry

The registry is a uthash hash table (open-addressing, header-only library) keyed on metric name. Each entry is a `metric_registry` struct:

Listing 2: Registry entry

```

1 typedef struct {
2     char    key[64];           // metric name (hash key)
3     HeadBlock *head;          // in-memory buffer
4     ChunkMetadata *chunks;     // dynamically grown array
5     int64_t  chunkCount;
6     int16_t  chunkCapacity;
7     UT_hash_handle hh;        // uthash intrusive handle
8 } metric_registry;

```

All access to the global `registry` pointer is protected by `registry_lock` (a `PTHREAD_MUTEX_INITIALIZER` static). Per-head-block `pthread_mutex_t` lock fields allow concurrent reads and writes against *different* metrics without contending on the global lock.

4.2 Head Block

Each metric's most-recent unflushed data lives in a `HeadBlock`:

Listing 3: Head block struct

```

1 typedef struct {
2     long    *timestamps;      // parallel array, capacity HEAD_CAPACITY
3     double  *values;
4     int     size;
5     long    lastTimestamp;    // monotonicity guard
6     pthread_mutex_t lock;
7 } HeadBlock;

```

`HEAD_CAPACITY` is set to 50 000. A PUT appends in $O(1)$ time. If the incoming timestamp is less than `lastTimestamp` the write is rejected with an error (out-of-order enforcement). If the head block is

full, `Head_PUT` unlocks, calls `headflush`, re-locks, and retries the append — this is the only place where a `PUT` blocks on disk I/O.

4.3 Startup — Registry Discovery

When the server starts, `loadRegistry` scans the data directory. Every subdirectory corresponds to a metric; within each metric directory, every `.chunk` file is read via `chunkread` (header only) to recover the `start_ts` and `end_ts` values, which are stored in the `ChunkMetadata` array. WAL replay then follows for each metric.

4.4 Flushing

`headflush` (in `registry.c`) is the bridge between the storage engine and the compression layer:

1. Lock `head->lock`.
2. Build the chunk file path: `<dataDir>/<metric>/<metric>_<first_ts>.chunk`.
3. Call `flushtochunk` (compression layer) with the head block's timestamp and value arrays.
4. On success, append a new `ChunkMetadata` entry, reset `head->size` and `head->lastTimestamp` to 0, and call `wal_truncate`.
5. Unlock.

`flushtochunk` creates a `bitwriter`, initialises both encoders, loops over the N points calling `tsencoderwrite` and `valencoderwrite` in tandem, closes the writer, fills a `chunkheader`, and calls `chunkwrite`.

4.5 Query Execution

4.5.1 GET

`Head_GET` iterates the `ChunkMetadata` array and decompresses every chunk whose `[start_ts, end_ts]` interval overlaps `[from, to)`. Results from each chunk are concatenated into a dynamically grown string. The head block is then scanned linearly for in-range points. The merged string is returned to the client.

4.5.2 AGG

`Head_AGG` uses a two-phase approach:

1. **Collection:** all points in range from disk chunks and the head block are gathered into a flat `Point[]` array (heap-allocated, dynamically resized).
2. **Sort:** `qsort` by timestamp ($O(N \log N)$).
3. **Reduce:** a single linear pass groups points into `[bstart, bstart+bucket)` windows and computes `avg/min/max/sum/count`.

If the downsampling feature is active and the query qualifies (`range` ≥ 3600 s, `bucket` ≥ 60 s, coarse directory exists), the collection phase reads from the `<metric>_1m/` coarse directory instead of raw chunks.

5. Chunk File Format

5.1 Binary Layout

A chunk file begins with a fixed 28-byte header followed by the compressed payload (a single concatenated bitstream for both timestamps and values) and ends with 8 bytes of reserved space for a future CRC-64 footer. All multi-byte integers use little-endian byte order.

Table 1: Chunk file binary layout

Offset	Size (B)	Field	Description
0	4	<code>chunkid</code>	First timestamp cast to <code>uint32</code> , used as file-name component
4	8	<code>startts</code>	First timestamp in chunk (<code>uint64</code>)
12	8	<code>endts</code>	Last timestamp in chunk (<code>uint64</code>)
20	4	<code>sizebytes</code>	Byte length of compressed payload (<code>uint32</code>)
24	4	<code>point_count</code>	Number of encoded points (<code>uint32</code>)
28	<i>sizebytes</i>	Payload	Timestamp bitstream followed immediately by value bitstream
<i>28+sizebytes</i>	8	(future)	Reserved for CRC-64 integrity footer

5.2 Payload Structure

Both the timestamp encoder and the value encoder write into the *same* `bitwriter` instance. The two logical streams are therefore concatenated in the payload buffer: all N timestamp codes appear first, immediately followed by all N value codes. The decoder reads exactly N timestamps then exactly N values using the `point_count` field as the terminator — no length separator is needed between the two streams.

5.3 Naming Convention

Chunk files live in per-metric subdirectories:

```
<dataDir>/<metric>/<metric>_<first_ts>.chunk
e.g.    ./data/cpu.usage/cpu.usage_1728000000.chunk
```

The directory is created on first flush. The `chunkid` field in the header equals `(uint32_t)first_ts`, providing a fast way to sort chunks chronologically by filename.

5.4 Compression Arithmetic

For N points on a perfectly regular 1-second metric:

$$\text{Timestamp bits} = \underbrace{64}_{\text{first}} + \underbrace{14}_{\text{second}} + \underbrace{(N-2)}_{\text{rest, all } D=0} \times 1 = N + 76 \quad (1)$$

$$\text{Value bits (constant)} = \underbrace{64}_{\text{first}} + \underbrace{(N-1)}_{\text{XOR=0}} \times 1 = N + 63 \quad (2)$$

For $N = 50\,000$:

$$\text{Total bits} = 50,076 + 50,063 = 100,139 \approx 12,518 \text{ bytes}$$

$$\text{Naive} = N \times 16 = 800,000 \text{ bytes} \quad \Rightarrow \quad \text{Ratio} \approx \frac{800,000}{12,518 + 28} \approx 63.7\times$$

For a slowly drifting random walk (each value costs ≈ 13 bits from the “reuse block” XOR branch), the ratio falls to $\approx 8\text{--}12\times$, consistent with the Gorilla paper’s production figures.

6. Bitstream I/O

6.1 Design Goals

The compression schemes produce variable numbers of bits per encoded value — as few as 1 bit for a repeated timestamp or value, up to 64 bits for a raw literal. Byte-aligned I/O is therefore useless;

we need a library for reading and writing *individual bits* into a byte buffer.

6.2 Bit Writer (bwwrite)

The writer maintains:

- `struct bytebuffer *buffer` — a heap-allocated, dynamically resizing `uint8_t` array.
- `uint8_t current_byte` — the partial byte being built.
- `int filledbits` — how many bits of `current_byte` have been written (0–7).

`bwwrite(bw, value, n_bits)` appends the low `n_bits` of `value` *most-significant-bit first*. It loops, taking as many bits from `value` as fit in the remaining space of `current_byte`, flushing to the buffer when the byte fills.

`bwclose` flushes any partial byte with its low bits set to zero (padding), then returns. The padding zeros are never read back because the decoder is driven by an explicit point count, not by stream exhaustion.

6.3 Bit Reader (brrread)

The reader is the exact mirror image:

- `struct bytebuffer *buffer` — pointer to the payload byte array.
- `int byteoffset` — index of the byte currently being consumed.
- `int bitsconsumed` — how many bits of that byte have been read.

`brrread(br, n_bits)` reassembles the next `n_bits` from the buffer, MSB-first, into a `uint64_t`. Reading past the end of the buffer returns the accumulated partial result (this can only happen due to padding zeros, which the decoder ignores).

6.4 Round-Trip Guarantee

Because both writer and reader process bits MSB-first and advance through the byte buffer in the same order, an arbitrary sequence of `bwwrite(value, n)` calls followed by a matching sequence of `brrread(n)` calls returns the original values exactly. The unit test `test_bit_io.c` verifies this with 1000 random values at random bit widths (1–64).

7. Delta-of-Delta Timestamp Encoding

7.1 Intuition

Consider a metric sampled every 10 seconds. Its raw timestamps are $t_1, t_1 + 10, t_1 + 20, t_1 + 30, \dots$. Storing them raw costs $N \times 64$ bits. Storing the deltas $\delta_i = t_i - t_{i-1}$ costs $N \times 64$ bits (same, if deltas are 64-bit). Storing the *differences between deltas*, $D_i = \delta_i - \delta_{i-1}$, costs mostly 1 bit per value because $D_i = 0$ whenever the sampling interval is constant. This is delta-of-delta (DoD) encoding.

7.2 Encoder State Machine

The encoder (`timestamp.c`) keeps three fields: `prevtimestamp`, `prevdelta`, and two boolean flags `isfirst/issecond`. Transitions:

Call #	Action	Bits written
1st	Write t_1 as raw 64-bit integer	64
2nd	Write $\delta_1 = t_2 - t_1$ as 14-bit unsigned	14
≥ 3 rd	Compute $D = \delta_i - \delta_{i-1}$, encode with prefix code	1, 9, 12, 16, or 36

7.3 Variable-Length Prefix Codes for D

The magnitude of D selects a prefix and a payload width:

Table 2: Delta-of-delta encoding table

Condition	Prefix bits	Payload bits	Total	Frequency
$D = 0$	0	0	1	> 99% for regular data
$-64 \leq D \leq 63$	10	7	9	rare jitter
$-256 \leq D \leq 255$	110	9	12	uncommon
$-2048 \leq D \leq 2047$	1110	12	16	rare
otherwise	1111	32	36	very rare

The payload is stored as a two's-complement signed integer in the specified number of bits. This is identical to the encoding in Gorilla Section 4.1.1.

7.4 Decoder

The decoder reads 1 bit at a time until it finds a 0 or accumulates four 1s, then reads the appropriate payload width and sign-extends via `signextend(val, bits)`. It reconstructs $\delta_i = D + \delta_{i-1}$ and $t_i = t_{i-1} + \delta_i$.

7.5 Worked Example — Timestamps

Input sequence: $t_1 = 1728000000$, $t_2 = 1728000010$, $t_3 = 1728000020$, $t_4 = 1728000030$, $t_5 = 1728000031$

Step 1 — First timestamp (64 bits)

Write $t_1 = 1,728,000,000$ as raw 64-bit little-endian integer.

Hex: 0x0000000066E6B980

Bits emitted: 64

Step 2 — First delta (14 bits)

$\delta_1 = t_2 - t_1 = 10$

Write 10 as 14-bit unsigned: 00 0000 0000 1010

Bits emitted: 14 (running total: 78)

Step 3 — t_3 : DoD = 0 (1 bit)

$\delta_2 = t_3 - t_2 = 10$

$D = \delta_2 - \delta_1 = 10 - 10 = 0$

Write prefix 0

Bits emitted: 1 (running total: 79)

Step 4 — t_4 : DoD = 0 (1 bit)

$\delta_3 = t_4 - t_3 = 10$

$D = 10 - 10 = 0$

Write prefix 0

Bits emitted: 1 (running total: 80)

Step 5 — t_5 : DoD = -9 (9 bits)

$$\delta_4 = t_5 - t_4 = 1$$

$$D = 1 - 10 = -9 \quad (-64 \leq -9 \leq 63 \Rightarrow 9\text{-bit case})$$

Two's complement of -9 in 7 bits: $128 - 9 = 119 = 111\,0111$

Write prefix 10 then payload 111 0111

Bits emitted: 9 (running total: 89)

Summary:

- Total bits: $64 + 14 + 1 + 1 + 9 = 89$ bits = 12 bytes (padded)
- Naive (5 timestamps $\times$ 8 bytes): 40 bytes
- Compression ratio on this sample: $\approx 3.3\times$

At scale ($N = 50,000$ all-regular points): $64 + 14 + 49,998 = 50,076$ bits = 6,260 bytes vs. 400 000 bytes naive = $63.9\times$.

8. XOR Value Encoding**8.1 Intuition**

Two IEEE 754 doubles representing similar measurements differ in only a handful of mantissa bits. XOR-ing them yields a 64-bit integer with long runs of zero bits at both ends and a small “meaningful block” in the middle. By recording only the leading-zero count, the meaningful-bit count, and the meaningful bits themselves, we avoid storing the constant zero stretches.

8.2 Low-Nibble Zeroing

Before encoding, the implementation zeroes the low 4 bits of the double's IEEE 754 representation:

```
1 valbits &= 0xFFFFFFFFFFFFFOULL;
```

This trades a tiny precision loss ($< 2^{-48}$ relative error) for significantly improved XOR compressibility: random low-mantissa noise that would otherwise prevent block reuse is eliminated.

8.3 Encoder State

The encoder tracks:

- `prevvaluebits` — IEEE 754 bits of the last encoded value.
- `prevleading` — leading zero count of the last non-zero XOR.
- `prevtrailing` — trailing zero count of the last non-zero XOR.

8.4 Encoding Cases

Table 3: XOR value encoding cases

Case	Condition	Overhead bits	Total
Zero XOR	$X = 0$ (identical value)	$0 = 1$ bit	1
Reuse block	Leading/trailing fit inside previous block	$10 + n$ meaningful	$2 + n$
New block	Otherwise	$11 + 5 + 6 + n$ meaningful	$13 + n$

For the “reuse block” case, $n = 64 - \text{prev_leading} - \text{prev_trailing}$ meaningful bits are written starting at bit position **prev_trailing** (LSB side). The decoder knows exactly where to place them because it keeps the same state. For the “new block” case, 5 bits encode the new leading-zero count (capped at 31) and 6 bits encode the new meaningful-bit count (value 0 means 64). Both **prevleading** and **prevtrailing** are updated.

8.5 Worked Example — Values

Input sequence: $v_1 = 45.0$, $v_2 = 45.0$, $v_3 = 46.5$

IEEE 754 representations (after low-nibble mask):

Step 1 — First value (64 bits)

Write $\text{bits}(v_1) = 0x4046800000000000$ as raw 64-bit integer.

Bits emitted: 64

Step 2 — $v_2 = v_1$: XOR = 0 (1 bit)

$X = 0x4046800000000000 \oplus 0x4046800000000000 = 0$

Write 0

Bits emitted: 1 (running total: 65)

Step 3 — $v_3 = 46.5$: new block (17 bits)

$X = 0x4047400000000000 \oplus 0x4046800000000000 = 0x0001C00000000000$

Binary of X :

0000 0000 0000 0001 1100 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000

Leading zeros: **15** Trailing zeros: **46** Meaningful bits: $64 - 15 - 46 = 3$

No previous block $\Rightarrow$ **new block** branch:

Write 11 (2 bits)

Write leading count 15 = 01111 (5 bits)

Write meaningful count 3 = 000011 (6 bits)

Meaningful bits of X (shifted right 46): $0x0001C00000000000 \gg 46 = 7 = 111$ (3 bits)

Update: **prev_leading** = 15, **prev_trailing** = 46

Bits emitted: 16 (running total: 81)

Summary:

- Total bits: $64 + 1 + 16 = 81$ bits = 11 bytes (padded)
- Naive (3 doubles $\times$ 8 bytes): 24 bytes
- Ratio: $\approx 2.2\times$ on 3 values; grows to $> 60\times$ with constant-value segments at scale.

8.6 Decoding

The decoder mirrors the encoder state exactly. On each call:

1. Read 1 bit. If 0: XOR = 0, return previous value.
2. Read 1 more bit. If 0: read n meaningful bits using **prev_leading/prev_trailing**, shift left by **prev_trailing**, XOR with previous bits.
3. If 1: read 5-bit leading, 6-bit meaningful length, then meaningful bits. Update **prev_leading/prev_trailing**. Reconstruct value.

The sign-extend step from the timestamp decoder is not needed here because XOR values are always positive integers (bitmasks).

9. Benchmark Results

9.1 Test Environment and Configuration

Table 4: Benchmark configuration

Parameter	Value
Number of metrics	10 (m0–m9)
Points per metric	50 000
Total points	500 000
Timestamp base	Unix 1 728 000 000 (Oct 4 2024)
Timestamp interval	1 second (perfectly regular)
Value pattern	Constant per segment, 2 step changes per metric
Batch size (PUT pipeline)	2 000 commands per flush
Head block capacity	50 000 (one flush per metric)
Build type	Release (-O2)

9.2 Ingestion Throughput

The benchmark client pipelines PUT commands in batches of 2 000 before waiting for acknowledgements. This keeps the server’s TCP receive buffer full and hides round-trip latency.

Table 5: Ingestion throughput results

Metric	Measured	Requirement	Margin
Total points inserted	500 000	—	—
Wall-clock time	≈2.5 s	—	—
Throughput	≈200 000 pts/s	≥50 000 pts/s	4×
Points acknowledged	500 000	—	—

The server sustains ≈200 000 ingestions per second, four times the required 50 000. The bottleneck is TCP loopback latency, not the storage engine; profiling shows the `PUT_value` call itself takes <1 μs per point.

9.3 Compression Results

Table 6: Disk compression results

Metric	Value	Note
Naive format (16 B/pt × 500 000)	8 000 000 B (7.63 MB)	naive.bin
Chunk files on disk (10 files)	≈125 600 B (123 KB)	Gorilla compressed
Chunk header overhead (10 × 28 B)	280 B	Fixed cost
Compression ratio	≈63.7×	Requirement: ≥8×
Timestamp bitstream (per metric, 50 000 pts)	≈6 260 B	DoD encoding
Value bitstream (per metric, 50 000 pts)	≈6 251 B	XOR encoding
Total per chunk	≈12 539 B	

9.4 Query Latency

Table 7: Query latency after full benchmark load (500 000 points)

Command	Query	Points scanned	Latency
GET	m0, first 10 points	10	< 1 ms
GET	m0, full range	50 000	< 10 ms
AGG avg	m0, 1000-s buckets, full range	50 000	< 15 ms
STATS	m0	—	< 1 ms

9.5 Compression Ratio Analysis

Why our ratio ($\approx 63.7\times$) exceeds the theoretical best ($\approx 11\text{--}12\times$): The Gorilla paper’s 1.37 bytes/point figure assumes a *slowly drifting random walk* where consecutive values differ by a small amount — each XOR uses the “reuse block” branch (≈ 13 bits). Our benchmark data uses *constant-value segments*: within each of the three segments, every value is identical to its predecessor (XOR = 0, 1 bit per value). This is the single-bit minimum and is unrealistically good. Similarly, our timestamps are perfectly regular (every $D = 0, 1$ bit), which is the minimum possible cost. Real-world metrics with nanosecond jitter or variable sampling intervals would show occasional $D \neq 0$ values costing 9 or 12 bits. The $63.7\times$ ratio is therefore a **best-case benchmark**.

10. Conclusion

This implementation demonstrates that it is possible to build a performant, durable time-series database from scratch in $\approx 2\,500$ lines of C. The key results are:

- **Ingestion:** $\approx 200\,000$ points/second, 4 *times* above the 50 000 pts/s requirement.
- **Compression:** $\approx 63.7\times$ on constant-segment benchmark data; $\approx 8\text{--}12\times$ on realistic slowly-drifting data, meeting the $\geq 8\times$ requirement.
- **Correctness:** all nine unit test suites pass; the full end-to-end round-trip (insert $\rightarrow$ flush $\rightarrow$ query) reproduces every inserted value exactly.
- **Durability:** the WAL ensures no acknowledged PUT is lost across a crash.
- **Manageability:** per-metric retention policies automatically enforce data lifetimes; downsampling reduces long-range query cost by up to $60\times$.

The bit-level implementation of the Gorilla encoders was the most challenging aspect of the project. Off-by-one errors in the variable-bit encoding (writing 7 bits when 9 were intended) produced symptoms that only appeared after the first few correctly-decoded values, requiring careful isolation of each component.